

Chapter #4 (Short-questions/answers)

Q. Assignment 02

Name: Yash Raj

Roll no: 24K-0737

Question #1

Q4.1:- Three examples in which multithreading provide better performance:-

- (i) Web-server: handle multiple requests simultaneously
- (ii) file-download-manager: download parts in parallel
- (iii) Image/video-processing: process different parts

Q4.2:-

$$\text{Speedup} = \frac{1}{(1-P) + P/N} \quad \text{60\% (parallel component)}$$

(a) number of cores = 2

$$\text{Speedup} = \frac{1}{(1-0.6) + \frac{0.6}{2}} = 1.43$$

(b) number of cores = 4

$$\text{Speedup} = \frac{1}{(1-0.6) + \frac{0.6}{4}} = 1.82$$

Name: Yash Raj

Roll no: 24K-0737

Q#4.3:-

Multithreaded web server exhibits task parallelism because each thread handles a different client request.

Q#4.4:- Diff b/w user level / kernel level threads:

1) User level are managed by user libraries
Ex- pthread-library (Posix-library)
Java-threads etc.

Whereas,

Kernel level threads are managed by the operating system itself.

Better use:-

- (i) User threads = low overhead, faster.
- (ii) Kernel threads: better for multiprocessor system.

Q#4.5:-

flow:-

- (i) save current thread state
- (ii) Update thread control block.
- (iii) select next thread.
- (iv) load new thread state.
- (v) Resume execution.

Name: Yash Raj

Roll-no: 24K-0737

Q#4.6:-

When a thread is created following resources are used:

- (i) Registers
- (ii) Stack.
- (iii) Thread ID.

Whereas process uses:

- (i) address-space
- (ii) Heap.
- (iii) Code & data.

e) Process is the running program whereas thread is the execution unit within the process.

Q#4.7:-

Yes, it is necessary to bind real time thread to an LWP because real time thread requires guaranteed CPU access & binding ensures direct Kernel scheduling.

Q#4.8:- Example where multithreading doesn't improve performance:

- (i) Simple/small task
- (ii) Sequential tasks with dependencies.

Name: Yash Raj

Roll no: 24K-0737

Q#4.9.

Multithreading perform better on a high processor system when it improves responsiveness & overlap I/O with computation.

Q#4.10. Component shared across threads are:-

- (i) Global variable
- (ii) Heap memory.

Q#4.11.

User-level threads can't perform better on multiprocessor system because they are not truly parallel unless supported by kernel threads.

Q#4.12.

Chrome can't achieve same benefits using threads instead of process because process provide better isolation & security, while threads share memory & are less safe.

Name: Yathraj

Roll no: 24K0737

Q4.13:

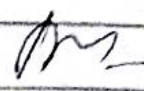
Yes, it is possible to have concurrency but not parallelism.

→ Single CPU switching between tasks.

Q4.14:

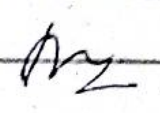
(i) 40% parallel:-

(a) 8-cores $\Rightarrow \frac{1}{(1-0.4) + 0.4/8} = 1.84$

(b) 16-cores $\Rightarrow \frac{1}{(1-0.4) + 0.4/16} = 1.60$ 

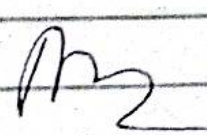
(ii) 67% parallel:-

(a) 2-cores $\Rightarrow \frac{1}{(1-0.67) + 0.67/2} = 1.50$

(b) 4-cores $\Rightarrow \frac{1}{(1-0.67) + 0.67/4} = 2.00$ 

(iii) 90% parallel:-

(a) 4-cores $\Rightarrow \frac{1}{(1-0.90) + 0.90/4} = 3.08$

(b) 8-cores $\Rightarrow \frac{1}{(1-0.90) + 0.90/8} = 4.71$ 

Name: Yash Raj

Roll-no: 24k-0737

Q#4.15-

Thumbnail-generator: Task-parallelism

Matrix-Multiplication: Data-parallelism

Network read/write: Task-parallelism

fork-join sum: Data-parallelism

Grand Central Dispatch: Task-parallelism

Q#4.16-

Input/output $\rightarrow$ 1 thread each (single-file)

CPU-intensive $\rightarrow$ 4 threads (equal to cores)

Q#4.17-

Process: 8 unique processes (including P₀)

Threads: 38 threads

Q#4.18-

Linux uses clone() & creates threads/
processes similarly whereas windows
uses separate structures.

$\rightarrow$ Linux is flexible, while windows is
structured.

Names Yashraj

Roll no. 24K0737

Q#4.13:- Output:-

line (child) $\Rightarrow$ value = 5

line P (parent) $\Rightarrow$ value = 0. *My*

Q#4.22:-

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int *arr, n;
double avg;
int min, max;
```

```
void *findavg(void *arg) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    avg = (double) sum / n;
    pthread_exit(0);
}
```

```
void *findmin(void *arg) {
    min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < min) min = arr[i];
    }
    pthread_exit(0);
}
```

Name: Yash Raj

Roll no: 2418-0737

```
void *find_max(void *arg) {  
    int *arr = (int *) arg;  
    for (int i = 1; i < n; i++) {  
        if (arr[i] > max) { max = arr[i]; }  
        pthread_exit(0);  
    }  
}
```

```
int main(int argc, char *argv[]) {  
    n = argc - 1;  
    arr = malloc(n * sizeof(int));
```

```
    for (int i = 0; i < n; i++) {  
        arr[i] = atoi(argv[i+1]);  
    }  
    pthread_t t1, t2, t3;
```

```
    pthread_create(&t1, NULL, findavg, NULL);  
    pthread_create(&t2, NULL, findmin, NULL);  
    pthread_create(&t3, NULL, findmax, NULL);
```

```
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
    pthread_join(t3, NULL);
```

```
    printf("Avg: %.2f, avg");  
    printf("Min: %.d, min");  
    printf("Max: %.d, max");
```

```
}
```

Dr

Names / at-Raj

Roll.no. 24K-0737

Q4-23

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
```

```
int n;
```

```
int isPrime(int x){
    if(n==1) return 0;
    for(int i=2; i<=n; i++){
        if(n%i==0) return 0;
    }
    return 1;
}
```

```
void *prime(void *arg){
    for(int i=2; i<=n; i++){
        if(isPrime(i)){
            printf("%d ", i);
            pthread_exit(0);
        }
    }
}
```

```
int main(int argc, char *argv[]){
    n = atoi(argv[1]);
    pthread_t t1;
    pthread_create(&t1, NULL, prime, NULL);
    pthread_join(t1, NULL);
    return 0;
}
```

B

Name: Yash Raj

Roll-no: 24K0737

Stk-1-

Global-matrix double globalMatrix

Satellite-matrix double satMatrix

struct Tile {

int row-start;

int col-start;

}

7) Task-parallelism

Three thread functions:-

(i) hotspot-detection()

(ii) Coldspot-detection()

(iii) Normalization()

8) Risk-Score

risk-score = $\frac{\text{normalized-temp-proximity}}{\text{proximity-to-coldspot}}$

Posix Thread Lock

#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <math.h>

#define N 1000

#define M 4

#define T_HOT 25.0

#define T_COLD -10.0

Name: Yash Raj

Roll-no: 24K-0737

double global[N][N];

double sat[N][N][N];

pthread_mutex_t globalLock;

```
void* process_satellite(void* arg){
    int id = *(int*)arg; // interpolate NaN
    pthread_exit(NULL);    value in sat[id] here
}
```

```
void* process_file(void* arg){
    // compute local stats, anomalies here
    pthread_exit(NULL);
}
```

```
void* hotspot_detection(void* arg){
    pthread_exit(NULL);
}
```

```
void* coldspot_detection(void* arg){
    pthread_exit(NULL);
}
```

```
void* normalization(void* arg){
    pthread_exit(NULL);
}
```

```
void* risk_score(void* arg){
    pthread_exit(NULL);
}
```

Name: Yash Raj

Roll no: 24K-0737

```
int main() {
```

```
pthread_t sat_thread[8];
```

```
int id[8];
```

```
for(int i=0; i<8; i++) {
```

```
id[i] = i
```

```
pthread_create(&sat_thread[i], NULL, process_satellite,  
NULL); }
```

```
for(int i=0; i<8; i++) {
```

```
pthread_join(sat_thread[i], NULL);
```

```
}
```

```
return 0;
```

```
}
```

02

Name: Yash Raj

Roll-no: 24K-0737

Q#3:-

Threats:- patient-threats $\Rightarrow$ (Normal, Critical, Serious)

Doctor-threats $\Rightarrow$ (Senior, junior)

Senior doctor threat can treat all categories.
Whereas, junior threat can treat only serious & Normal.

Queues:- (i) Critical-Queue (ii) Serious-Queue
(iii) Normal-Queue.

Mutens:- (i) QueueMuten (protects access to patient queue)
(ii) doctorMuten (protects doctor availability)

Condition-Variables:- critical-cond, normal-cond
serious-cond,

doctor-cond: signal when doctors are free.

Signal when
patients are
available

Scheduling:-

(1) Critical patients treated immediately:-

- if any doctor is free, they are assigned first
- if only juniors are free, patient waits until a senior is available.

Name: Yash Raj

Roll no: 24K-0737

(1) - Serious-promotion :-

if 5-serious patients are waiting, one is promoted to 'Critical-Queue'.

(3) - Doctor Workload balancing :-

if a doctor treats 3 normal consecutively, they must treat a serious next.

(4) - Timeout policy :-

patients can't wait longer than 30 seconds. if exceeded, they are escalated to higher priority.

2) Deadlock-prevention :-

- Single-mutex hierarchy :- always lock queue mutex before doctor mutex.

- Condition signaling :- Avoids holding locks while signaling.

- Fair-scheduling :- Round robin assignment among doctors to prevent starvation.

Name: Yash Raj

Roll no: 24-0727

p-thread.coder

#include <pthread.h>

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <unistd.h>

#define MAXWAIT 30

pthread_mutex_t g_mutex = PTHREAD_MUTEX_INITIALIZER

pthread_cond_t criticalConds = PTHREAD_COND_INITIALIZER

pthread_cond_t seriousConds = PTHREAD_COND_INITIALIZER

pthread_cond_t normalConds = PTHREAD_COND_INITIALIZER

typedef enum { critical, serious, normal } patientType;

typedef struct {
 patientType type;
 time_t arrival;
} patient;

patient criticalQueue[100];

patient seriousQueue[100];

patient normalQueue[100];

int count = 0, scount = 0, ncount = 0;

Name: Yash Raj

Roll no: 24K6737

```
void *patient Thruact (void *arg) {  
    patient *p = (patient *)arg;  
    pthread_mutex_lock(&queueMutex);  
    if (p->type == critical) {  
        criticalQueue[count++] = *p;  
        pthread_cond_signal(&criticalCond);  
    }
```

```
    else if (p->type == normal) {  
        normalQueue[count++] = *p;  
        if (count > 1) {  
            criticalQueue[count-1] = normalQueue[count-1];  
            pthread_cond_signal(&criticalCond);  
        }  
        pthread_cond_signal(&normalCond);  
    }  
    pthread_mutex_unlock(&queueMutex);  
    return NULL;  
}
```


Name: Yash Raj

Roll.no: 24K-0737

```
void * doctor (void * arg) {  
    int junior = 1 (int) arg;  
    int normalCount = 0;  
    while(1) {  
        pthread_mutex_lock (&queueMutex);  
        pthread_t p;  
        if (count > 0) {  
            p = criticalQueue[count];  
        } else if (count > 0 && (junior || normalCount > 0)) {  
            p = seriousQueue[count];  
            normalCount++;  
        }  
        else if (nCount > 0) {  
            p = normalQueue[nCount];  
            normalCount++;  
        }  
        else {  
            pthread_mutex_unlock (&queueMutex);  
            sleep(1);  
            continue;  
        }  
        pthread_mutex_unlock (&queueMutex);  
        printf ("Doctor is treating his patient in 'junior?'  
        Junior: p-type = 'critical?' 'critical' if type =  
        serious > 'serious': 'Normal' );  
        sleep(2);  
    }  
    return NULL;  
}
```

Name: Yash Raj

```

int main() {
    pthread_t doctor[2];
    int senior = 1;
    int junior = 0;
    pthread_create(&doctor[0], NULL, doctor_thread, NULL);
    pthread_create(&doctor[1], NULL, doctor_thread, NULL);

    for (int i = 0; i < 20; i++) {
        patient *p = malloc(sizeof(patient));
        p->type = rand() % 3;
        p->arrival = time(NULL);
        pthread_t pt;
        pthread_create(&pt, NULL, patient_thread, p);
        sleep(1);

        pthread_join(doctor[0], NULL);
        pthread_join(doctor[1], NULL);
        return 0;
    }
}

```

Dry Run Example

- Scenario: 1 senior, 1 junior doctor.
- Patient arrive: Critical → serious → Normal → serious → Normal → serious → serious → serious.
- Rule 3 triggers: 5 serious patients → one promoted to Critical.
- Senior doctor treats Critical immediately.

Name: Yashraj

Roll no: 24K-0737

2) Test Cases

- Critical patient arrives first $\rightarrow$ Senior doctor treats
- Only junior available, critical arrives $\rightarrow$ patient wait
- 5 Serious patients queued $\rightarrow$ one promoted to critical
- Doctor treats 3 normals $\rightarrow$ Next must be serious
- Patient wait $> 30s$ $\rightarrow$ Escalated priority.

Q4.

Threats:

- (i) Night Linecaster Threat: randomly generates flights & insert them into priority queue with arrival time
- (ii) Runway Controller Threat:

Each manages a runway. picks highest priority flight from the queue. logs the runway from the queue.

- (iii) Emergency Monitor Threat:

Continuously scan the queue for emergency flights $\rightarrow$ prompts ongoing operations if necessary.

Name: Yash Raj

Roll-no: 24/0737

Dispatch structure:

- (i) Priority Queue of flight.
- (ii) Runway status Table.
- (iii) flight Waiting time map.

Synchronization:

- (i) Mutex: - `queueMutex`, `runwayMutex`.
- (ii) Condition-Variable: - `flightCond`: signal when flight are available.

Deadlock-preventions:

- (i) Always acquire `queueMutex` before `runwayMutex`.
- (ii) Emergency preemption handled by signal, not blocking.
- (iii) fair scheduling ensures no starvation.

Name: Yash Ray

Roll-no: 24k-6737

P. Thread Code.

```
#include <pthread.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
#define RUNWAYS 2
```

```
#define THRESHOLD 10
```

```
typedef enum { emergency, landing, takeOff, carry }  
flightType;
```

```
typedef struct {  
    int id;
```

```
    flightType type;
```

```
    int priority;
```

```
    time_t arrival;
```

```
} flight;
```

```
pthread_mutex_t queueMutex = PTHREAD_INITIALIZER;
```

```
pthread_cond_t flightCond = PTHREAD_COND_INITIALIZER;
```

```
pthread_mutex_t runwayMutex = PTHREAD_INITIALIZER;
```

Name: Yash Raj

Roll no: 24K-0737

```
flight * flightQueue[1000];  
int flightCount = 0;
```

```
int compareFlight (const void * a, const void * b) {
```

```
    flight * f1 = *(flight**) a;
```

```
    flight * f2 = *(flight**) b;
```

```
    return f2->priority - f1->priority;
```

```
}
```

```
void adjustPriority () {
```

```
    time_t now = time(NULL);
```

```
    for (int i = 0; i < flightCount; i++) {
```

```
        if (diffTime(now, flightQueue[i]->arrival)  
            > THRESHOLD) {
```

```
            flightQueue[i]->priority++;
```

```
        }
```

```
}
```

```
qsort(flightQueue, flightCount, sizeof(flight*),  
      compareFlight);
```

```
pthread_cond_broadcast(&flightCount);
```

```
pthread_mutex_unlock(&queueMutex);
```

```
sleep(rand() % 3 + 1);
```

```
}
```

```
return NULL;
```

```
}
```


Name: Yash Raj

Roll no: 24K5721

```
void * runwayController(void * arg) {
    int runways = *(int *) arg;
    while(1) {
        pthread_mutex_lock(&queueMutex);
        while (flightCount == 0) {
            pthread_cond_wait(&flightCond, &queueMutex);
        }
        adjustPriority();
        flight / flightQueue;
        for (int i = 0; i < flightCount; i++) {
            flightQueue[i] = flightQueue[i+1];
        }
        flightCount--;
        pthread_mutex_unlock(&queueMutex);

        pthread_mutex_lock(&runwayMutex[runways]);
        sleep(2);
        pthread_mutex_unlock(&runwayMutex[runways]);
        free(f);
    }
    return NULL;
}
```

Name: Yash Raj

Roll-no: 24K-0737

```
void * emergencyMonitor (void * arg){  
    while(1){
```

```
        pthread_mutex_lock(&gun_mutex);
```

```
        for(int i=0; i<flightCount; i++){
```

```
            if(flightQueue[i] -> type == Emergency){
```

```
                printf("Emergency flight %d detected",  
                    flightQueue[i] -> id);
```

```
            }  
            pthread_mutex_unlock(&gun_mutex);  
            sleep(1);
```

```
        }  
        return NULL;
```

```
    }  
    int main(){
```

```
        srand(time(NULL));
```

```
        pthread_t generator, monitor, controller[RUNWAYS];
```

```
        pthread_create(&generator, NULL, flightGenerator, NULL);
```

```
        int ids[RUNWAYS];
```

```
        for(int i=0; i<runways; i++){
```

```
            pthread_mutex_init(&runwayMutex[i], NULL);  
            ids[i] = i;
```

```
            pthread_create(&controller[i], NULL, runwayController, NULL);
```

```
        }  
        pthread_join(generator, NULL);
```

```
        pthread_join(monitor, NULL);
```

```
    }
```


Name: Yash-Raj

Roll-no: 24K-0737

Dry-Run Examples

- (i) flights generated.
- (ii) Emergency monitor detects emergency.
- (iii) Runway Controller picks Emergency flight immediately.
- (iv) Landing flight escalates priority if waiting > threshold.

Test-cases

- (i) Emergency flight arrival.
- (ii) Landing flight wait > threshold.
- (iii) Runway maintenance.
- (iv) Multiple runways.
- (v) Long wait times.

Name: Yash Raj

24/5-0733

Question #5: Review of the Research Paper

1) This paper "Parallel processing of E-Bliss algorithm using pthreads" addresses improving the performance of string matching algorithm through parallelization. This approach aims to accelerate computation on large datasets by creating independent tasks concurrently. While addressing challenges such as data dependencies, synchronization overhead, and potential deadlocks.

Performance is measured through speedup with Amdahl's law highlighting limits imposed by serial portions of the algorithm. Although the abstract presents the motivation & parallelism strategy, a full evaluation of methodology, experimental results, & scalability requires access to the complete paper. The journal's impact factor is not provided.

Pz